A Project Report on

GRIDWISE DATABASE SYSTEM

Relational Database Architecture and Enforcement Engine for Microgrid Energy Scheduling

CSE 207 : Database System

SUBMITTED BY

Name	Student ID	Intake
Md Taimur Rahman	20255103046	55
Md Showkotul Islam	20255103058	55
Md Yousuf Ali	20255103059	55
Md Siam Khan	20255103069	55

Abstract

Modern campus microgrids combine utility power, rooftop solar generation and battery storage, and every 24-hour dispatch plan for such a system must obey strict physical and operational rules. In most student database projects these rules live only in application code, so any direct SQL client or batch script can silently store an invalid plan. GridWise is a relational database system for smart-campus microgrid energy scheduling that moves this enforcement into the database engine itself. The schema gridwise_db consists of 10 tables normalized to Boyce-Codd Normal Form, declarative PRIMARY KEY, FOREIGN KEY, UNIQUE and CHECK constraints, 11 views for derived metrics and violation reporting, 6 triggers, 7 stored procedures and 1 stored function. BEFORE triggers verify the hourly energy balance, solar limits, battery rate and capacity limits, state-of-charge continuity, end-of-day battery neutrality and operator directives, raising SQLSTATE 45000 for any invalid row, while AFTER triggers write an append-only audit trail.

The database is loaded with 70 scenarios (10 public reference cases and 60 synthetic cases), 137 plans and 3,288 hourly dispatch records, all of which pass the validation views. An automated test harness (sp_selftest) runs 18 positive and negative tests, 26 analytical queries demonstrate joins, CTEs, window functions and EXPLAIN-based index analysis, and a PHP PDO REST API with a single-page HTML dashboard exposes the data through XAMPP. The project demonstrates that a well-designed DBMS can act as the active guardian of business rules rather than a passive data store.

Keywords: *relational database, BCNF, triggers, stored procedures, data integrity, microgrid, energy scheduling, MariaDB, PHP, REST API.*

List of Figures

Figure 1: Three-tier system architecture of GridWise 7
Figure 2: Use case diagram 7
Figure 3: Entity-Relationship (ER) diagram of gridwise_db 8
Figure 4: Data Flow Diagram - Level 0 (context diagram) 9
Figure 5: Data Flow Diagram - Level 1 10
Figure 6: Flowchart of row validation by triggers 13
Figure 7: Dashboard screen with KPI cards and the directive usage chart 24
Figure 8: Scenario list with source filter and search 25
Figure 9: Scenario details: battery data, operator notes and input chart 25
Figure 10: Plan inspector with totals, hourly chart and rule check 26
Figure 11: Import console with a template case 27
Figure 12: Successful import of a valid case 27
Figure 13: Rejected case: trigger error and automatic rollback 28
Figure 14: The How it works guide panel 28
Figure 15: Dashboard in dark theme 29

Table of Contents

Acknowledgement i
Abstract ii
Table of Contents iii
List of Figures v
List of Tables vi
Chapter 1: Introduction 1
1.1 Problem Specification 1
1.2 Objectives 1
1.3 Motivation and Scope 2
1.4 Project at a Glance 2
1.5 Organization of the Project Report 3
Chapter 2: Background 4
2.1 Existing System Analysis 4
2.2 Supporting Literature 4
2.3 Microgrid Domain Concepts 5
Chapter 3: System Analysis & Design 6
3.1 Technology & Tools 6
3.2 Model & Diagram 6
3.2.1 Model (SDLC/Agile/Waterfall/OOM) 6
3.2.2 System Architecture 6
3.2.3 Use Case Diagram 7
3.2.4 Class Diagram / ER Diagram 8
3.2.5 Data Flow Diagram 9
3.2.6 Database Schema 11
3.2.7 Algorithms / Flowchart 11
3.3 Features, Functional and Non-functional Requirements 14
3.3.1 Features 14
3.3.2 Functional Requirements 14
3.3.3 Non-functional Requirements 15
Chapter 4: Implementation 16
4.1 Interface / Front-End Design 16
4.2 Back-End 16
4.3 Modules / Features 17
4.4 Testing and Results 19
4.4.1 Automated Self-Test (sp_selftest) 19
4.4.2 Universal Compliance Audit 20
4.4.3 Savings Against the No-Battery Baseline 20
4.4.4 Rejection of Invalid Data and Rollback 20
Chapter 5: User Manual 21
5.1 System Requirement 21
5.1.1 Hardware Requirement 21

5.1.2 Software Requirement 21
5.2 Installation and Setup 21
5.2.1 Step 1: Start the Services 21
5.2.2 Step 2: Copy the Web Application 21
5.2.3 Step 3: Check the Database Connection 22
5.2.4 Step 4: Import the Database 22
5.2.5 Step 5: Verify the Installation 23
5.2.6 Step 6: Open the Dashboard 23
5.3 User Interfaces 23
5.3.1 Dashboard (KPI Cards and Charts) 23
5.3.2 Scenario List 24
5.3.3 Scenario Details and Plan Inspector 25
5.3.4 Import a Case (API Console) 26
5.3.5 Help Panels and Themes 28
5.3.6 Login Credentials 29
5.4 Temporary Live Preview for Remote Demonstration 29
5.5 Troubleshooting 30
Chapter 6: Conclusion **30**
6.1 Conclusion 31
6.2 Limitations 31
6.3 Future Works 32
References **33**
Appendix A: Project Files, Tests and Code **33**
A.1 Project directory structure 34
A.2 Automated self-test cases (sp_selftest) 34
A.3 Rule violation raised by the validator 35
A.4 Selected analytical queries 35
A.5 Transaction and rollback demonstration 36
Appendix B: Data Dictionary **37**

CHAPTER 1

INTRODUCTION

1.1 Problem Specification

Modern educational campuses are operated as small electrical systems. A typical campus draws power from the public utility grid at hourly tariffs that vary between off-peak, normal and peak periods, produces energy from rooftop solar panels during daylight, and may keep a centralized battery energy storage system (BESS) that can be charged when electricity is cheap and discharged when it is expensive. Scheduling this system for the next 24 hours produces a **dispatch plan**: for every hour it states how much energy is imported from the grid, how much solar is used and whether the battery charges, discharges or stays idle.

A dispatch plan is only useful if it is physically valid. For every hour the energy balance must hold, solar usage cannot exceed what the panels can produce, the battery cannot be charged or discharged faster than its rate limits, and its stored energy must stay between a reserve minimum and its capacity. In addition, operators issue textual advisories (panel cleaning, charger outages, relay protection windows, feeder caps) that further restrict the system.

Typical undergraduate database projects (student records, library catalogs, inventory trackers) use the database only as a passive container, and all validation lives in application code. When another program, a batch script or a person with direct SQL access bypasses that code, invalid data is stored without any warning. The problem addressed in this project is therefore: **how to design a relational database in which no mathematically or physically invalid energy plan can ever be committed to disk, no matter which client writes to it.**

1.2 Objectives

The objectives of the GridWise Database System are:

- To design a normalized relational schema (up to BCNF) that models scenarios, battery hardware, hourly inputs, operator notes, directives, dispatch plans and audit logs.
- To enforce the physical and operational rules of a microgrid inside the DBMS using CHECK constraints, foreign keys, triggers and stored procedures.
- To derive all aggregates (total cost, total grid energy, peak grid import, savings) through views instead of storing redundant data.
- To keep an automatic, append-only audit trail of every change to dispatch plans.
- To prove correctness with a self-test suite of 18 positive and negative tests, a transaction/rollback demonstration and a plan compliance audit.
- To demonstrate intermediate and advanced SQL (joins, subqueries, CTEs, window functions, full-text search and EXPLAIN-based index analysis) through 26 analytical queries.

- To provide a lightweight PHP and HTML web dashboard with a REST API so that the database can be explored and tested by evaluators.

At the end of the project these objectives are compared with the measured results in Section 6.1.

1.3 Motivation and Scope

Motivation

- Moving domain rules into the DBMS makes data integrity independent of any single application, which is the central idea of a database course.
- Microgrid scheduling is a realistic and mathematically rich domain. It can be expressed completely as tables, constraints, triggers and views, which makes it an excellent case study for relational database design.
- Smart campus energy management reduces electricity cost, and a trustworthy data layer is the first requirement of any later optimizer or forecasting tool.

Scope

- **Included:** schema design and normalization, views, triggers, procedures, a function, seed data (10 reference and 60 synthetic scenarios), 26 analytical queries, a self-test harness, a transaction demonstration, a REST API and a web dashboard running on XAMPP.
- **Not included:** a plan optimizer or solver (plans are supplied and validated, not computed), live meter or IoT integration, user authentication and role management, and production deployment to a cloud database.
- **Target platform:** XAMPP on a local machine (MariaDB 10.4+ or MySQL 8.0+, PHP 7.4+ or 8.x) and a modern web browser.

1.4 Project at a Glance

Table 1 summarizes the final deliverable. Every figure was measured directly from the installed database (`gridwise_db`) and is reproduced by the commands given in Chapter 5.

Table 1: *GridWise Database System in numbers*

Item	Count	Item	Count
Normalized tables (BCNF)	10	Scenarios	70
Views	11	Public reference cases	10
Triggers	6	Synthetic cases	60
Stored procedures	7	Dispatch plans	137
Stored functions	1	Hourly dispatch records	3,288
Analytical queries	26	Operator notes / active directives	130 / 108
Automated self-tests	18	Plans passing validation	137 of 137

1.5 Organization of the Project Report

This report is organized into six chapters followed by references and appendices.

- **Chapter 1 - Introduction** states the problem, objectives, motivation and scope.
- **Chapter 2 - Background** reviews existing approaches to schedule management, the theory (normalization, constraints, triggers, transactions) and the microgrid terms used in the project.
- **Chapter 3 - System Analysis & Design** presents the tools, development model, architecture, use case, ER, data flow and schema designs, the validation algorithm and the requirements.
- **Chapter 4 - Implementation** explains the interface, the back-end (database layer and REST API), the modules, and the testing results.
- **Chapter 5 - User Manual** lists the requirements, installation steps and a guided tour of every screen of the user interface.
- **Chapter 6 - Conclusion** summarizes the findings, limitations and future work.
- **References and Appendices** list the sources, the directory structure, the 18 self-tests, selected code and the data dictionary.

CHAPTER 2
BACKGROUND

2.1 Existing System Analysis

Campus and building energy schedules are currently handled in three common ways. Each has strengths and weaknesses relevant to this project, summarized in Table 2.

Table 2: *Comparison of existing approaches with GridWise*

Approach	Strengths	Weaknesses
Manual / spreadsheet scheduling	No infrastructure cost; easy for facility staff to understand and edit.	No enforcement of rules (a formula can be overwritten); no audit history; no multi-user consistency; poor scalability beyond a few scenarios.
Application-validated CRUD systems	Familiar and fast to build; friendly validation messages.	The database trusts the application. Direct SQL access, import scripts or a second application can store invalid data, and every rule must be re-implemented in every client.
Commercial EMS / SCADA platforms	Live metering, optimization engines and dashboards.	Expensive and closed; not suitable as a teachable model of how integrity rules are expressed in relational form.
GridWise (this project)	Rules enforced inside the DBMS for every client; automatic audit trail; derived metrics through views; free, lightweight stack.	Validates plans but does not compute optimal plans; single-battery, hourly model.

GridWise takes the strengths of these approaches (a simple data model and a readable dashboard) and removes their main weakness by making the relational engine itself the authority on validity.

2.2 Supporting Literature

The project applies the following theoretical and technological knowledge from the Database System course:

- **Relational model and normalization (1NF to BCNF):** repeating groups such as 24 hourly values are separated into child tables, and derived values are excluded from tables to avoid update anomalies.
- **Declarative integrity:** PRIMARY KEY, FOREIGN KEY with ON DELETE CASCADE, UNIQUE and CHECK constraints reject illegal domain values at the schema level.
- **Procedural integrity:** BEFORE INSERT/UPDATE triggers call a validator procedure that raises SQLSTATE 45000 using SIGNAL, while AFTER triggers maintain an audit log.

- **Views:** eleven views encapsulate joins and aggregations so that queries, procedures and the web API share one definition of each metric.
- **Transactions and ACID:** transactions, SAVEPOINT and ROLLBACK ensure that a rejected row leaves the database unchanged.
- **Advanced SQL:** common table expressions, window functions (RANK, ROW_NUMBER, LAG, SUM OVER), anti-joins with NOT EXISTS, full-text search and EXPLAIN-based index analysis.
- **Web technologies:** a PHP PDO REST API and an HTML5/CSS3/JavaScript single-page dashboard on an Apache and MariaDB (XAMPP) stack were chosen because they are free, easy to install on any lab computer and need no external services.

2.3 Microgrid Domain Concepts

Table 3 defines the domain terms that appear throughout the report and in the database schema.

Table 3: *Glossary of microgrid terms used in the project*

Term	Meaning in GridWise
Scenario	One 24-hour planning problem: battery hardware, hourly demand, solar forecast, tariff and operator notes.
Demand (kWh)	Energy the campus consumes in a given hour.
Solar forecast (kWh)	Maximum energy the rooftop panels can supply in a given hour.
Tariff (BDT/kWh)	Price of grid electricity in a given hour (off-peak, normal or peak).
BESS / battery	Battery energy storage system with a capacity, an initial and minimum energy level, and maximum charge and discharge rates per hour.
Dispatch plan	For each of the 24 hours: grid import, solar used and battery action (charge, discharge or idle).
Directive	A structured rule extracted from an operator note, for example solar_reduction, no_charge_window or max_grid_window.
State of charge	Energy stored in the battery after an hour; it must stay between the active minimum and the capacity.
Baseline plan	A reference plan that never uses the battery; used to compute the savings of other plans.

CHAPTER 3

SYSTEM ANALYSIS & DESIGN

3.1 Technology & Tools

The following tools and technologies were used to develop and run the project:

Table 4: *Technology stack*

Component	Technology / Version	Purpose
Database engine	MariaDB 10.4+ or MySQL 8.0+ (via XAMPP)	Tables, views, triggers, routines
Web server	Apache HTTP Server (XAMPP 8.0+)	Serves dashboard and API
Server language	PHP 7.4+ / 8.x with pdo_mysql	REST API (PDO)
Front-end	HTML5, CSS3, vanilla JavaScript, inline SVG charts	Single-page dashboard
Data tools	Python 3 (`build_seed.py`, `gen_synthetic.py`)	Convert samples and generate synthetic scenarios
Admin GUI	phpMyAdmin / MySQL command line	Import SQL, run queries
Diagrams	Mermaid (repository ER source), Graphviz	Design diagrams in this report
Version control	Git and GitHub	github.com/SaKin32/Gridwise_db
Optional sharing	Cloudflare Tunnel / localtunnel	Temporary live preview for viva

3.2 Model & Diagram

The design follows a database lifecycle approach with diagrams for the structure, the data flow and the validation logic. The sub-sections below describe the model and each diagram.

3.2.1 Model (SDLC/Agile/Waterfall/OOM)

The project was developed with an iterative Waterfall (database development lifecycle) model: requirement analysis of the microgrid rules, conceptual design (ER model), logical design and normalization, physical implementation (DDL, views, routines, triggers), data loading, testing and documentation. Each phase was reviewed and, when testing exposed a problem, the earlier design phase was revisited. This model suits a database project because the schema is the foundation of everything else and must be stable before the views, triggers and web layer are built on it.

3.2.2 System Architecture

GridWise follows a three-tier architecture (Figure 1). The browser dashboard talks to a PHP REST API, which uses PDO to run SQL on the database. All business rules are enforced in the data tier so that they apply to every client, whether it is the web API, phpMyAdmin or a script.

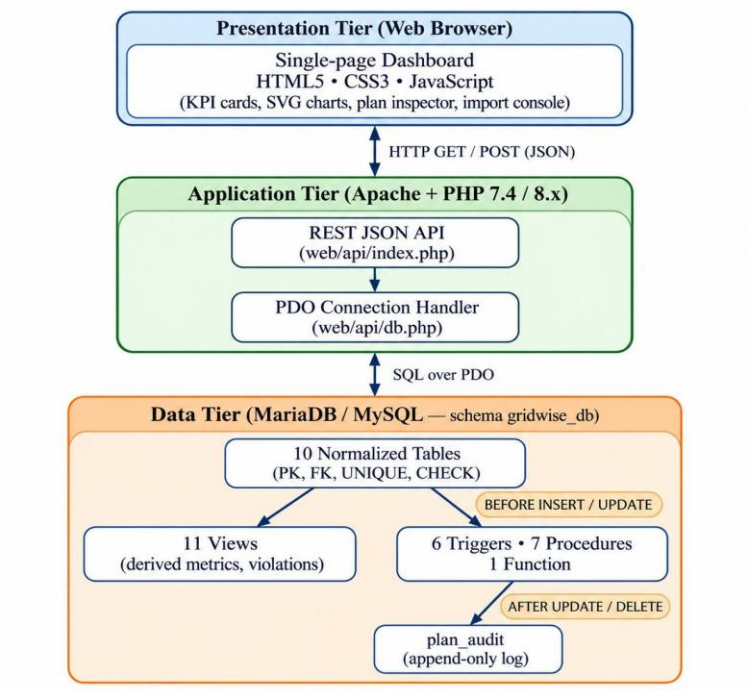


Figure 1: *Three-tier system architecture of GridWise*

3.2.3 Use Case Diagram

Two human actors interact with the system (Figure 2): the **campus operator or evaluator**, who browses, inspects, imports and validates cases, and the **database administrator**, who installs the database, runs queries and self-tests. Importing a case always includes automatic audit logging by the triggers.

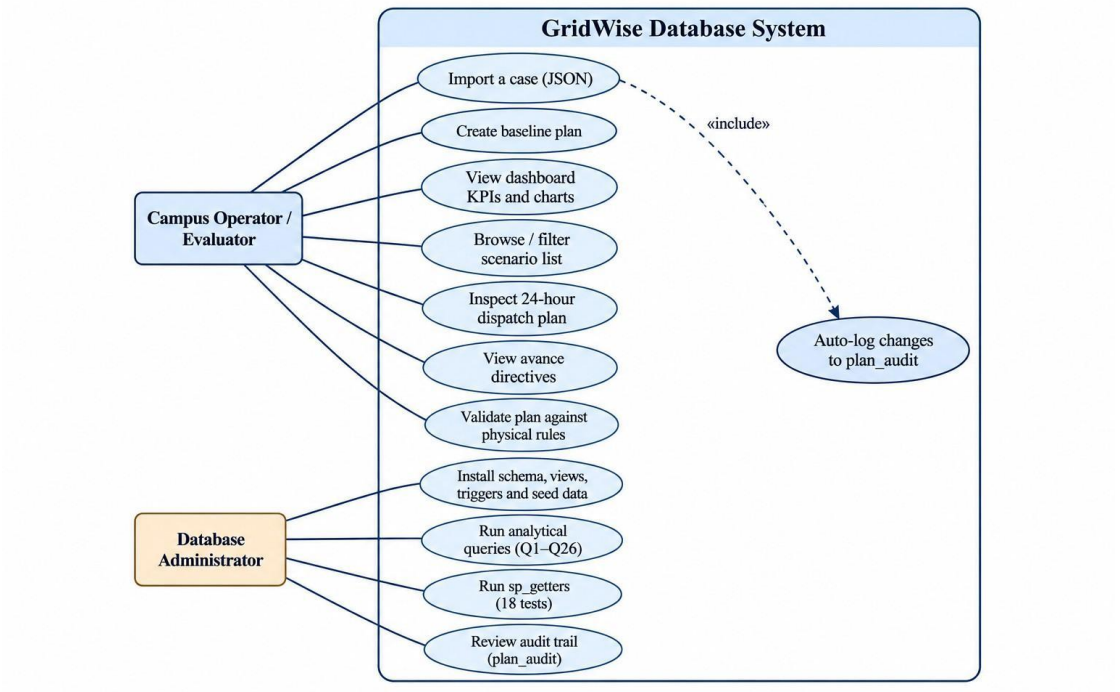


Figure 2: *Use case diagram*

3.2.4 Class Diagram / ER Diagram

The relational model decomposes the 24-hour problem into 10 tables in three tiers: the **problem tier** (scenario, battery_config, hourly_input), the **advisory tier** (operator_note, directive_type, directive, directive_hour) and the **execution tier** (plan, hourly_plan, plan_audit). Figure 3 shows the tables, their keys and the cardinality of every relationship.

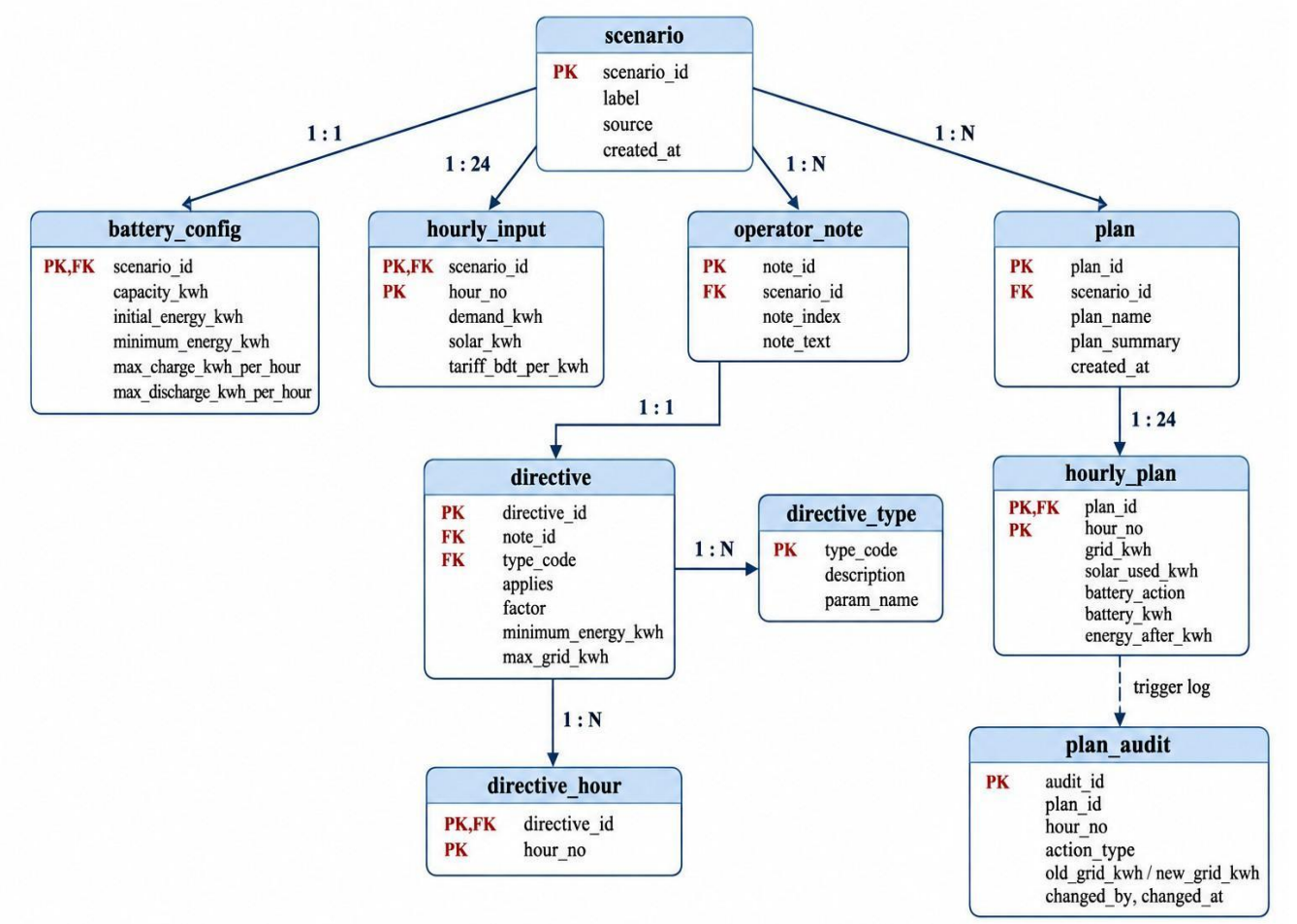


Figure 3: *Entity-Relationship (ER) diagram of gridwise_db*

Table 5 lists the relationships of the ER diagram.

Table 5: *Relationships between tables*

Parent table	Child table	Cardinality	Meaning
scenario	battery_config	1 : 1	Each scenario has exactly one battery.
scenario	hourly_input	1 : 24	One row per hour (0 to 23).
scenario	operator_note	1 : N	A scenario carries one or more operator notes.

Parent table	Child table	Cardinality	Meaning
operator_note	directive	1 : 1	Each note is interpreted as one structured directive.
directive_type	directive	1 : N	Lookup of the six directive types.
directive	directive_hour	1 : N	Hours in which a directive is active.
scenario	plan	1 : N	Several candidate plans per scenario.
plan	hourly_plan	1 : 24	A complete plan has 24 hourly rows.
hourly_plan	plan_audit	logged by trigger	Updates and deletions are recorded automatically.

3.2.5 Data Flow Diagram

The context-level diagram (Figure 4) shows the two external entities and the system as a single process. Level 1 (Figure 5) decomposes it into five processes and four data stores.

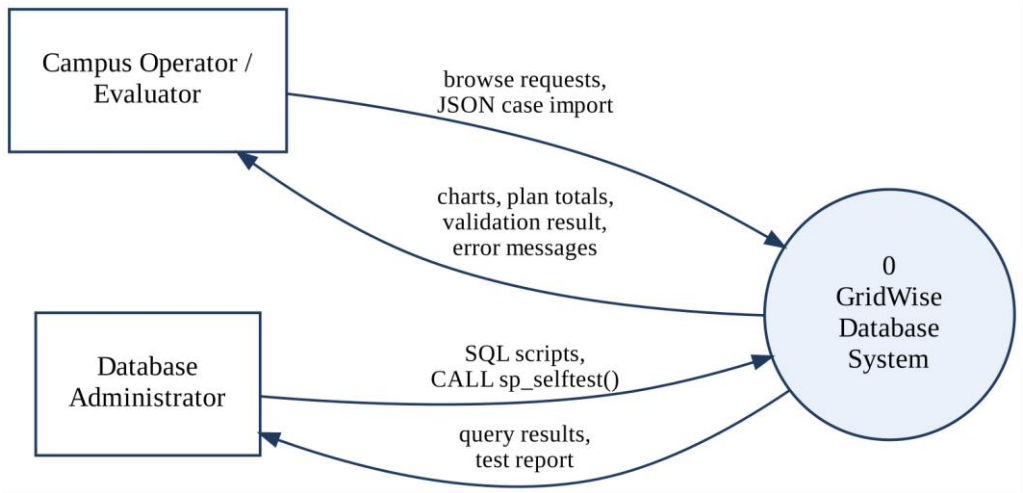


Figure 4: *Data Flow Diagram - Level 0 (context diagram)*

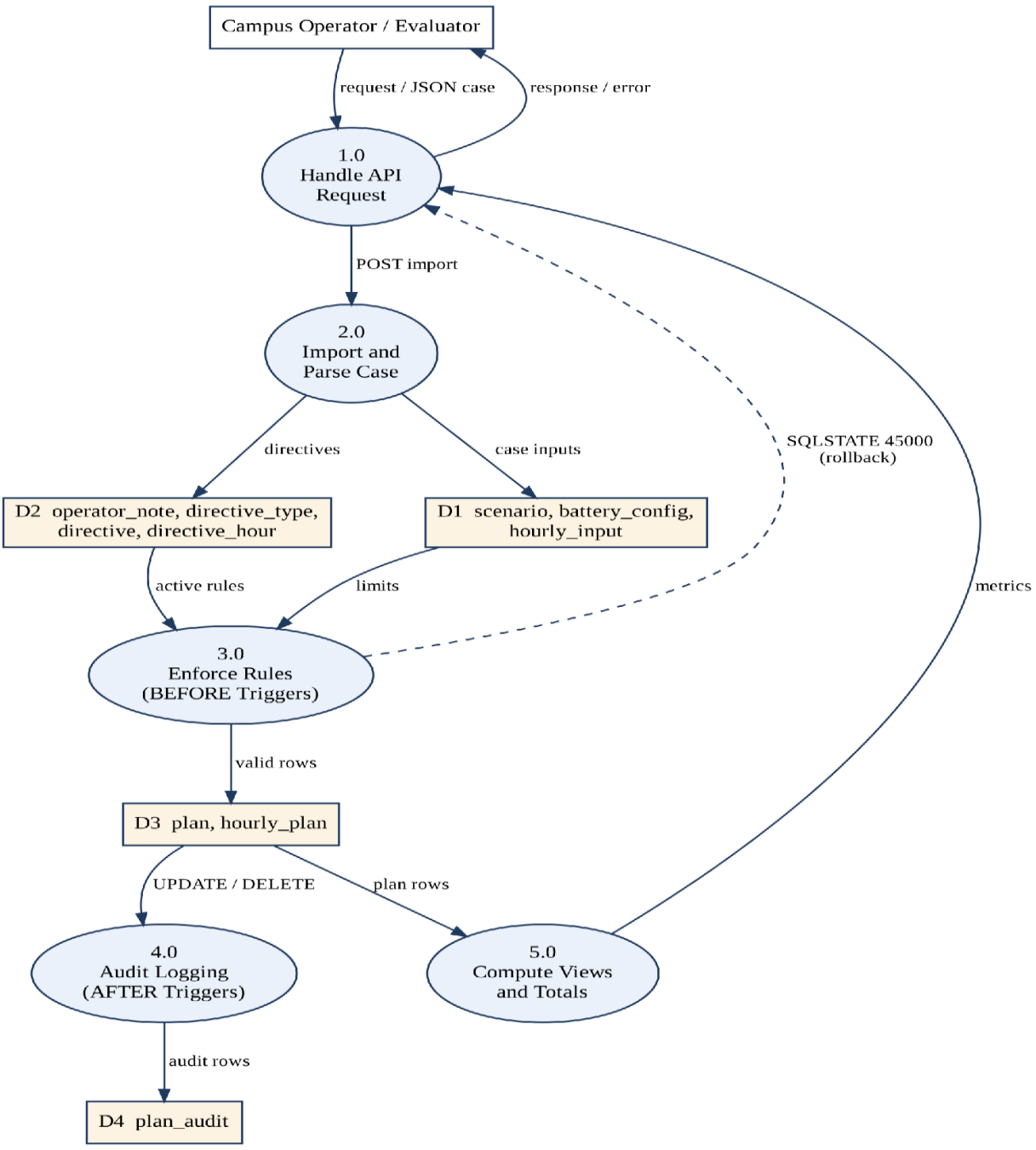


Figure 5: *Data Flow Diagram - Level 1*

3.2.6 Database Schema

The relational schema (see Figure 3 for relationships) is summarized in Table 6. All foreign keys use ON DELETE CASCADE except the lookup reference to directive_type. The full column-level data dictionary is given in Appendix B.

Table 6: *Relational schema summary*

Table	Keys	Other attributes
scenario	scenario_id (PK)	label, source (ENUM), created_at
battery_config	scenario_id (PK, FK)	capacity_kwh, initial_energy_kwh, minimum_energy_kwh, max_charge_kwh_per_hour, max_discharge_kwh_per_hour
hourly_input	scenario_id, hour_no (PK, FK)	demand_kwh, solar_kwh, tariff_bdt_per_kwh
operator_note	note_id (PK); scenario_id (FK)	note_index, note_text; UNIQUE(scenario_id, note_index)
directive_type	type_code (PK)	description, param_name
directive	directive_id (PK); note_id (FK, UNIQUE); type_code (FK)	applies, factor, minimum_energy_kwh, max_grid_kwh, explanation
directive_hour	directive_id, hour_no (PK, FK)	(associative table: directive to target hours)
plan	plan_id (PK); scenario_id (FK)	plan_name, plan_summary, created_at; UNIQUE(scenario_id, plan_name)
hourly_plan	plan_id, hour_no (PK, FK)	grid_kwh, solar_used_kwh, battery_action (ENUM), battery_kwh, energy_after_kwh
plan_audit	audit_id (PK)	plan_id, hour_no, action_type, old/new grid_kwh, old/new battery_action, changed_by, changed_at

Normalization. The tables satisfy BCNF: every attribute is atomic (1NF); in tables with composite keys no attribute depends on part of the key (2NF); there are no transitive dependencies, and totals such as total_cost_bdt, total_grid_kwh and peak_grid_kwh are deliberately not stored but computed by the view `v_plan_totals` (3NF); and every determinant is a candidate key (BCNF).

3.2.7 Algorithms / Flowchart

Every INSERT or UPDATE on `hourly_plan` runs the following algorithm inside the BEFORE trigger (procedure `sp_check_plan_row`). The core rules are:

Validation rules

1. **Energy Balance:** $grid + solar_used + discharge = demand + charge$ $(tolerance: 0.01kWh)$

2. **Solar Usage Constraint :**

$$0 \leq solar_used \leq solar_kwh \times directive_solar_factor$$

3. **Charge & Discharge Limits :**

$$charge \leq max_charge_kwh_per_hourdischarge \leq max_discharge_kwh_per_hour$$

4. **Hourly Energy State :** $energy_after[h] = energy_after[h-1] + charge - discharge$

   $(where\ hour\ 0\ starts\ from\ initial_energy_kwh)$

5. **Battery Capacity Constraints :**

$$minimum_energy(h) \leq energy_after[h] \leq capacity_kwh$$

6. **End-of-Day Energy Neutrality :**

$$energy_after[23] = initial_energy_kwh$$

7. **Directive Windows :**

$$DirectiveWindows: no_charge\ /\ no_discharge\ /\ max_grid_cap$$

Figure 6 shows the decision flow. The first rule that fails raises SQLSTATE 45000, the statement is rejected and the surrounding transaction can be rolled back. A valid row is stored and the AFTER triggers write the audit record.

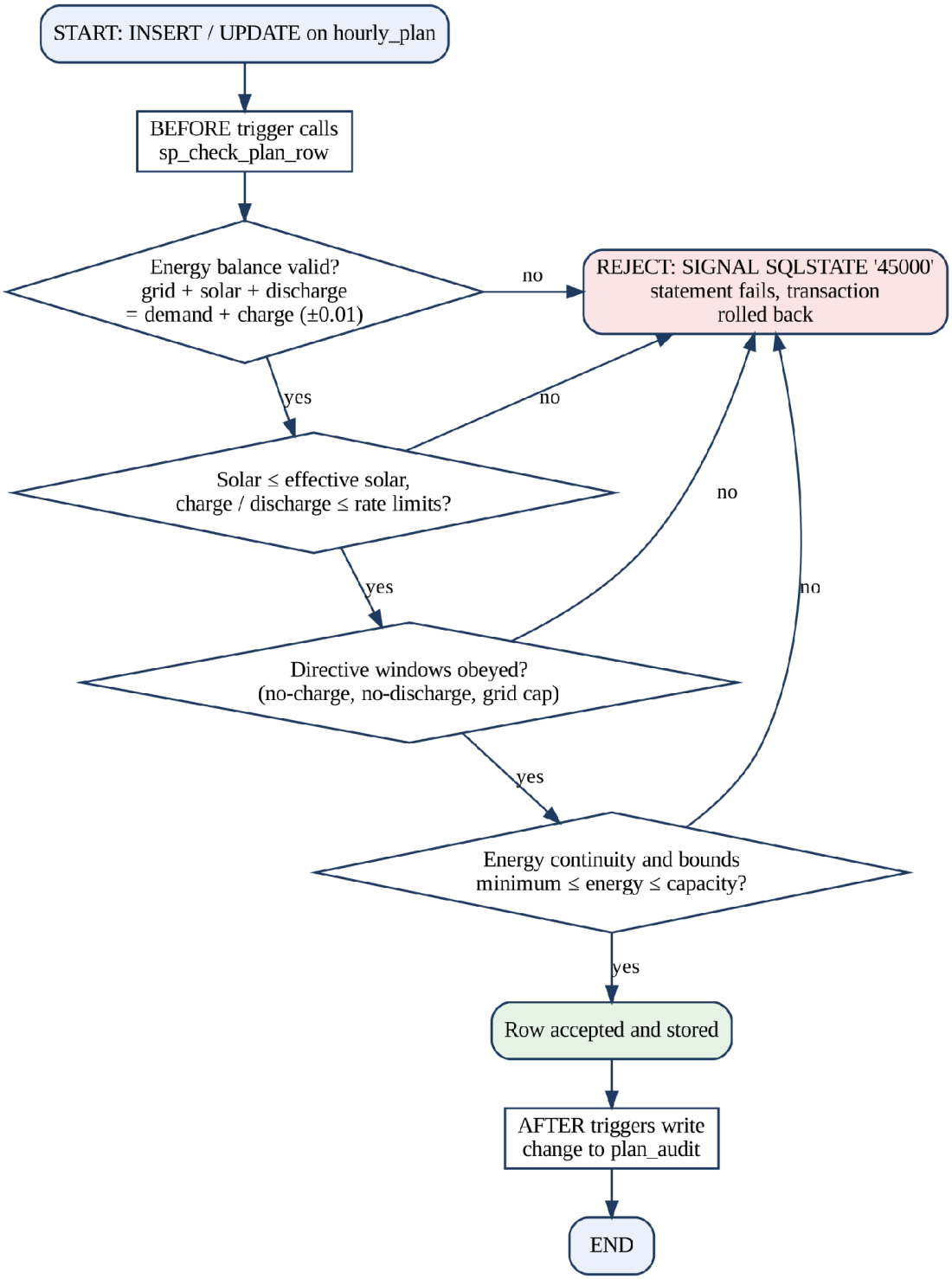


Figure 6: *Flowchart of row validation by triggers*

3.3 Features, Functional and Non-functional Requirements

The features and the functional and non-functional requirements of the system are listed below.

3.3.1 Features

- 10 BCNF-normalized tables with declarative PK, FK, UNIQUE and CHECK constraints.
- Real-time rule enforcement by triggers (SQLSTATE 45000) so invalid plans cannot be stored.
- Automatic append-only audit trail of updates and deletions in `plan_audit`.
- 11 views for effective solar, hourly constraints, plan totals, violations, status, savings versus a no-battery baseline and scenario overview.
- Stored procedures and a function: `sp_check_plan_row`, `sp_check_directive`, `sp_validate_plan`, `sp_plan_report`, `sp_add_directive`, `sp_create_baseline_plan`, `sp_selftest` and `fn_plan_cost`.
- 70 scenarios (10 public reference + 60 synthetic), 137 plans and 3,288 hourly records.
- 26 analytical queries and an 18-case automated self-test.
- Web dashboard (KPI cards, SVG charts, scenario browser, plan inspector, import console) with light and dark themes.

3.3.2 Functional Requirements

Table 7: *Functional requirements*

ID	Requirement
FR1	The system shall store scenarios with battery configuration and 24 hourly inputs (demand, solar, tariff).
FR2	The system shall store operator notes and their structured directives (solar_reduction, minimum_battery_reserve, no_charge_window, no_discharge_window, max_grid_window, no_op).
FR3	The system shall store dispatch plans with 24 hourly rows and reject any row that violates a rule.
FR4	The system shall validate a complete plan (`sp_validate_plan`) and report VALID or the list of violation codes.
FR5	The system shall compute plan cost, grid energy, peak import and savings against a no-battery baseline.
FR6	The system shall log every update and deletion of hourly plan rows with the old and new values and the database user.
FR7	The system shall provide a REST API to list scenarios, plans and directives and to import a JSON case atomically.
FR8	The system shall provide a dashboard to browse data and test the validation rules.

3.3.3 Non-functional Requirements

Table 8: *Non-functional requirements*

Attribute	Requirement
Integrity	Invalid data must be impossible to commit regardless of the client used.
Atomicity	A failed import must roll back completely and leave all tables unchanged.
Performance	Foreign-key columns are indexed and EXPLAIN is used to verify index use.
Portability	Runs on any XAMPP installation (Windows, Linux, macOS) without extra software.
Usability	A single-page dashboard with filters, charts and a built-in guide.
Maintainability	Modular, numbered SQL scripts and a unified `install_all.sql`.
Security	PDO prepared access layer; the database port is not exposed when sharing through a tunnel.

CHAPTER 4
IMPLEMENTATION

4.1 Interface / Front-End Design

The interface is a single HTML file (`web/index.html`) written in vanilla HTML5, CSS3 and JavaScript with no external libraries. It uses a collapsible sidebar with three working tabs (Dashboard, Scenario list, Import a case) and utility buttons (About Project, How it works, Light / Dark mode, Collapse menu). Charts (hourly demand, solar, tariff, dispatch stacks and battery state of charge) are drawn as inline SVG so that the dashboard works offline. A green badge shows whether the API is reachable, and every card can reveal the SQL query behind it, which supports the educational purpose of the project. A screen-by-screen description with screenshots is given in Section 5.3.

4.2 Back-End

The back-end has two layers. The **database layer** is created by the numbered scripts in `sql/` (schema, views, routines and triggers, seed data, queries, self-test and transaction demo). The **application layer** is a vanilla PHP REST API (`web/api/index.php`) that connects through PDO (`web/api/db.php`) and supports Create, Read, validation and Delete (user scenarios only) operations. The base endpoint is:

```
http://localhost/gridwise/api/index.php
```

Table 9: *REST API endpoints*

Method and action	Returns / behavior
GET ?action=health	Connectivity check: status ok and database version
GET ?action=scenarios	All scenarios from v_scenario_overview with invalid-plan counts
GET ?action=scenario&id=SAMPLE-01	Scenario, battery, 24-hour constraints, notes with directives, and its plans
GET ?action=plan&id=1	Full 24-hour dispatch schedule of a plan with totals
GET ?action=validate&plan_id=1	Runs sp_validate_plan and returns VALID or the violation list
GET ?action=stats	Counts, savings by plan type and directive usage for the dashboard
POST ?action=import_case	Atomically inserts a JSON case; if any trigger fires, the whole transaction is rolled back
POST ?action=baseline	Creates a no-battery baseline plan with sp_create_baseline_plan
POST ?action=delete_scenario	Deletes a scenario, only if it was imported by a user

Any endpoint can be tested from a terminal without the dashboard. The following command checks the connection and the response shows the status:

Command

```
curl "http://localhost/gridwise/api/index.php?action=health"
```

Response

```
{"status":"ok","db_version":"10.11.14-MariaDB"}
```

Validating a stored plan returns a JSON object. A valid plan reports the rule code `VALID`:

Command

```
curl "http://localhost/gridwise/api/index.php?action=validate&plan_id=1"
```

Response

```
{"plan_id":1,"valid":true,"result":[{"plan_id":1,"hour_no":null,
  "rule_code":"VALID","detail":"all rules satisfied"}]}
```

CRUD in the database. Create: INSERT into scenario, battery_config, hourly_input, operator_note, directive and plan (guarded by triggers). Read: views and the API endpoints. Update and Delete: allowed on `hourly_plan` only through the validating BEFORE UPDATE trigger, and always logged by the AFTER UPDATE and AFTER DELETE triggers.

4.3 Modules / Features

The project is divided into the following modules:

Table 10: *Project modules and files*

Module	File(s)	Description
Schema	`sql/01_schema.sql`	10 tables with keys, CHECK constraints and indexes
Views	`sql/02_views.sql`	11 analytical and validation views
Routines & triggers	`sql/03_routines_triggers.sql`	6 triggers, 7 procedures and fn_plan_cost
Reference data	`sql/04_seed_samples.sql`	10 public reference cases with reference plans
Synthetic data	`sql/05_seed_synthetic.sql`	60 generated scenarios (summer, winter, storm, peak-demand)
Queries	`sql/06_queries.sql`	26 queries Q01 to Q26 and an EXPLAIN index demo
Self-test	`sql/07_selftest.sql`	sp_selftest with 18 test cases
Transaction demo	`sql/08_transaction_demo.sql`	SAVEPOINT, ROLLBACK and audit demonstration
Installer	`sql/install_all.sql`	Runs scripts 01 to 05 in one import
Data tools	`tools/build_seed.py`, `tools/gen_synthetic.py`	Create seed SQL from JSON and generate synthetic cases
REST API	`web/api/index.php`, `db.php`	JSON endpoints over PDO
Dashboard	`web/index.html`	Single-page web interface
Documentation	`README.md`, `docs/`	Technical manual, ER diagram, normalization proofs

Triggers

Six triggers guard the two tables that carry rules. Four act on `hourly_plan` and two on `directive` (Table 11).

Table 11: *Triggers of gridwise_db*

Trigger	Event	Table	Purpose
trg_hplan_bi	BEFORE INSERT	hourly_plan	Calls sp_check_plan_row; rejects any rule-breaking row
trg_hplan_bu	BEFORE UPDATE	hourly_plan	Re-validates the modified row
trg_hplan_au	AFTER UPDATE	hourly_plan	Writes old and new values to plan_audit
trg_hplan_ad	AFTER DELETE	hourly_plan	Writes the deleted row to plan_audit
trg_directive_bi	BEFORE INSERT	directive	Calls sp_check_directive (guardrails on directive values)
trg_directive_bu	BEFORE UPDATE	directive	Re-validates the modified directive

Stored procedures and function

Table 12: *Stored routines*

Routine	Type	Purpose
sp_check_plan_row	Procedure	Validates one hourly row against all physical and directive rules
sp_check_directive	Procedure	Validates a directive and its reserve value
sp_validate_plan	Procedure	Checks a whole plan; returns VALID or the violation list
sp_plan_report	Procedure	Hour-by-hour report of a plan
sp_create_baseline_plan	Procedure	Builds a no-battery baseline plan for a scenario
sp_add_directive	Procedure	Adds a directive and its hours in one call
sp_selftest	Procedure	Runs the 18 automated tests (file 07_selftest.sql)
fn_plan_cost	Function	Returns the total cost (BDT) of a plan

Views

- `v_directive_hours`, `v_hour_constraints` (strictest-wins reconciliation of overlapping directives), `v_plan_rows`, `v_plan_totals`.
- `v_plan_violations` (11 failure modes), `v_plan_status`, `v_directive_summary`, `v_directive_issues`.
- `v_scenario_overview`, `v_no_battery_cost` and `v_plan_comparison` (savings in BDT and percent).

Analytical queries

Table 13: *Analytical query catalog (sql/06_queries.sql)*

Queries	Techniques demonstrated
Q01 - Q05	View queries, GROUP BY aggregates, conditional-aggregation JOIN, RANK() OVER (PARTITION BY)
Q06 - Q10	ROW_NUMBER, top-N with LIMIT, LEFT JOIN, HAVING, CASE aggregation
Q11 - Q13	Multi-table JOIN with GROUP_CONCAT, solar lost to reductions, battery use per plan type
Q14 - Q17	CTE with RANK, LAG, running SUM window, subquery against the average
Q18 - Q21	NOT EXISTS anti-join, FULLTEXT and LIKE search, tariff-band bucketing
Q22 - Q24	Compliance audits: rule violations, end-of-day battery level, directive structure (empty result = pass)
Q25 - Q26	Stored function fn_plan_cost and procedures sp_validate_plan / sp_plan_report
Index demo	EXPLAIN before and after `idx_input_tariff` (type=range versus full table scan)

4.4 Testing and Results

Correctness is demonstrated by four independent checks, all of which were executed on a freshly installed database for this report.

4.4.1 Automated Self-Test (sp_selftest)

The procedure inserts rule-breaking rows inside a transaction, checks that the expected error is raised, and ends with ROLLBACK so that data is unchanged. All 18 tests passed (Table 14). The test is started with:

SQL

```
CALL sp_selftest();
```

Table 14: *Result of sp_selftest (18 of 18 passed)*

No.	Test description	Verdict	Expected outcome
1	valid row is accepted (SAMPLE-01, hour 0, idle)	PASS	no error (accepted)
2	continuity: rejected if it does not match the previous hour	PASS	error: transition wrong
3	solar_reduction: using more than the reduced solar is rejected	PASS	error: solar_used exceeds
4	rejected when energy balance is broken	PASS	error: energy balance
5	rejected when charge rate limit is exceeded	PASS	error: charge exceeds
6	rejected when discharge rate limit is exceeded	PASS	error: discharge exceeds
7	rejected when battery goes above capacity	PASS	error: above capacity
8	rejected when battery goes below the minimum	PASS	error: below active minimum
9	rejected when battery does not return to initial in the last hour	PASS	error: end-of-day
10	no_charge_window: charging in a forbidden hour is rejected	PASS	error: no_charge_window

No.	Test description	Verdict	Expected outcome
11	no_discharge_window: discharging in a forbidden hour is rejected	PASS	error: no_discharge_window
12	max_grid_window: rejected when the grid cap is exceeded	PASS	error: max_grid_window
13	minimum_battery_reserve: rejected when going below the reserve	PASS	error: below active minimum
14	CHECK: battery_kwh > 0 while idle is rejected	PASS	error: ck_hplan_idle
15	guardrail: rejected when reserve exceeds capacity	PASS	error: reserve exceeds
16	CHECK: solar factor above 1 is rejected	PASS	error: ck_directive_factor
17	CHECK: applies=1 on no_op is rejected	PASS	error: ck_directive_applies
18	CHECK: giving a factor to no_charge_window is rejected	PASS	error: ck_directive_factor

4.4.2 Universal Compliance Audit

The view `v_plan_violations` lists every stored plan row that breaks any of the 11 rule types. After loading all 137 plans the result is an empty set:

SQL

```
SELECT COUNT(*) AS violations FROM v_plan_violations;
```

Result

```
+------------+
| violations |
+------------+
|          0 |
+------------+
```

4.4.3 Savings Against the No-Battery Baseline

The view `v_plan_comparison` derives cost and savings without storing any redundant value. For the first public reference cases the reference plans save between 9 and 10 percent compared with the baseline (Table 15).

Table 15: *Cost and savings from v_plan_comparison (first reference cases)*

Scenario	Plan	Total cost (BDT)	No-battery cost (BDT)	Savings (BDT)	Savings (%)
SAMPLE-01	reference_optimal	38,365.00	42,395.00	4,030.00	9.51
SAMPLE-01	baseline_no_battery	42,395.00	42,395.00	0.00	0.00
SAMPLE-02	reference_optimal	42,885.00	47,335.00	4,450.00	9.40
SAMPLE-02	baseline_no_battery	47,335.00	47,335.00	0.00	0.00

4.4.4 Rejection of Invalid Data and Rollback

When a case with a broken rule is imported through the API, the trigger raises SQLSTATE 45000, the API answers with HTTP 422 and the transaction is rolled back, leaving every table unchanged (screenshot in Figure 13). The same behavior is shown in SQL by `sql/08_transaction_demo.sql` (Appendix A.5).

CHAPTER 5

USER MANUAL

5.1 System Requirement

5.1.1 Hardware Requirement

Table 16: *Recommended minimum hardware*

Component	Requirement
Processor	Dual-core processor or better
Memory	4 GB RAM
Storage	About 1 GB free disk space for XAMPP and the database
Display	At least 1280 x 720 resolution
Network	Needed only for the optional temporary live preview

5.1.2 Software Requirement

Table 17: *Software requirements*

Component	Requirement
Operating system	Windows 10/11 (Linux or macOS also work with XAMPP)
Server stack	XAMPP 8.0 or later (Apache and MariaDB 10.4+ / MySQL 8.0+)
PHP	PHP 7.4+ or 8.x with the `pdo_mysql` extension enabled
Browser	A modern browser: Chrome, Firefox, Edge or Safari
Python 3	Only needed to regenerate the seed data (optional)

5.2 Installation and Setup

Follow these steps in order. Commands are shown in separate code boxes; type them exactly as written.

5.2.1 Step 1: Start the Services

Open the **XAMPP Control Panel** and click **Start** next to **Apache** and **MySQL**. The module names turn green and show their ports (80/443 for Apache and 3306 for MySQL).

5.2.2 Step 2: Copy the Web Application

Copy the contents of the `web/` folder of the project into the XAMPP web root so that the following three files exist afterwards:

Target location

```
C:\xampp\htdocs\gridwise\index.html
C:\xampp\htdocs\gridwise\api\index.php
C:\xampp\htdocs\gridwise\api\db.php
```

5.2.3 Step 3: Check the Database Connection

Open `api/db.php`. The default values match a fresh XAMPP installation and normally need no change:

Table 18: *Connection settings in api/db.php*

Setting	Default value
Host	127.0.0.1
Port	3306
Database	gridwise_db
User	root
Password	(empty)

5.2.4 Step 4: Import the Database

Method A - phpMyAdmin (graphical)

1. Open the following address in a browser:

```
http://localhost/phpmyadmin/
```

2. Click **Import**, choose the file `sql/install_all.sql` (format SQL) and click **Import** at the bottom of the page.
3. The left sidebar now shows `gridwise_db` with 10 tables and 11 views.
4. If the single file is too large for the upload limit, import these files one by one in this order: `01_schema.sql`, `02_views.sql`, `03_routines_triggers.sql`, `04_seed_samples.sql`, `05_seed_synthetic.sql`.

Method B - command line

Open Command Prompt in the `sql` folder of the project and run:

Windows Command Prompt

```
C:\xampp\mysql\bin\mysql.exe -u root -p < install_all.sql
```

The installer runs scripts 01 to 05. The self-test and the transaction demonstration are separate files; import them when needed:

Optional scripts

```
C:\xampp\mysql\bin\mysql.exe -u root -p gridwise_db < 07_selftest.sql
C:\xampp\mysql\bin\mysql.exe -u root -p gridwise_db < 08_transaction_demo.sql
```

5.2.5 Step 5: Verify the Installation

Run the following statements in phpMyAdmin (SQL tab) or in the MySQL client. They should return the numbers shown in Table 19.

SQL

```
USE gridwise_db;
SELECT COUNT(*) FROM scenario;
SELECT COUNT(*) FROM plan;
SELECT COUNT(*) FROM hourly_plan;
SELECT COUNT(*) FROM v_plan_violations;
CALL sp_selftest();
```

Table 19: *Expected results after installation*

Statement	Expected result
SELECT COUNT(*) FROM scenario;	70
SELECT COUNT(*) FROM plan;	137
SELECT COUNT(*) FROM hourly_plan;	3288
SELECT COUNT(*) FROM v_plan_violations;	0 (no rule violation)
CALL sp_selftest();	18 passed, 0 failed (after importing 07_selftest.sql)

5.2.6 Step 6: Open the Dashboard

Open the dashboard in a browser:

```
http://localhost/gridwise/index.html
```

A green **API ok** badge in the top-right corner confirms that the browser, PHP and the database are connected.

5.3 User Interfaces

The dashboard is a single page. The black **sidebar** on the left switches between screens, the **top bar** holds search, refresh and the connection badge, and a light purple **hint bar** under the title explains the current screen in one line. All screenshots in this section were taken from the running system with the installed data.

5.3.1 Dashboard (KPI Cards and Charts)

The Dashboard is the default screen. It loads automatically from the `stats` API and gives a live overview of the database (Figure 7).

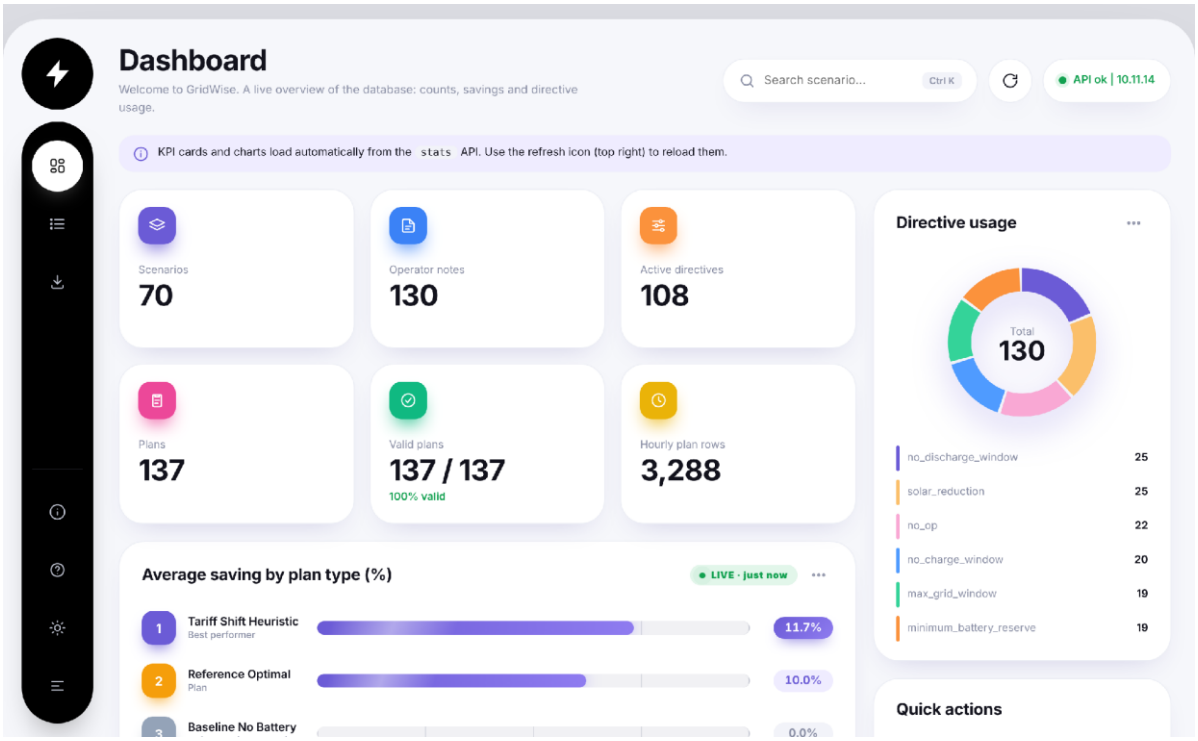


Figure 7: *Dashboard screen with KPI cards and the directive usage chart*

Table 20: *Elements of the Dashboard screen*

Element	Function
KPI cards	Six cards: Scenarios (70), Operator notes (130), Active directives (108), Plans (137), Valid plans (137 / 137, 100% valid) and Hourly plan rows (3,288).
Average saving by plan type	Ranked bars showing the average percentage saving of every plan type against the no-battery baseline. A LIVE badge shows when the data was last loaded.
Cost comparison chart	Compares the cost of reference and baseline plans for the 10 sample cases in BDT.
Directive usage	Donut chart with the number of directives per type and a legend with counts.
Card menu (three dots)	Shows or hides the SQL query behind the card.
Quick actions	Buttons that jump to the Scenario list, the Import console and the guide.
Refresh icon	Reloads the data of the current screen.
API badge	Green: API and database reachable (shows the database version). Red: connection problem.

5.3.2 Scenario List

The Scenario list shows all 70 scenarios in one table (Figure 8). The dropdown filters by source (public_sample, synthetic or user) and the search box filters by id or label. The columns show the number of notes, active directives and plans, and a status badge that turns from OK to a warning if any plan of the scenario is invalid.

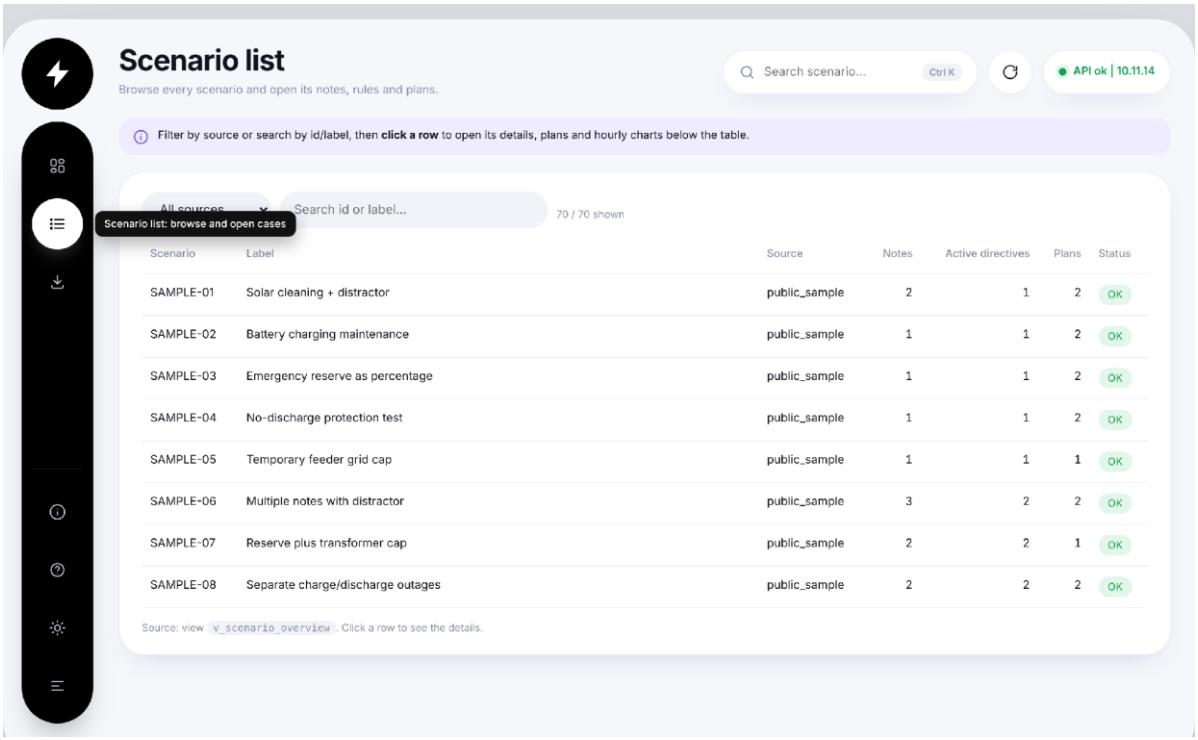


Figure 8: *Scenario list with source filter and search*

5.3.3 Scenario Details and Plan Inspector

Clicking a row opens the details below the table. The upper part (Figure 9) shows the battery parameters, the operator notes with their parsed directives (type, hours and value), and a chart of hourly demand, effective solar, tariff and the colored directive bands.



Figure 9: *Scenario details: battery data, operator notes and input chart*

The lower part is the **Plan inspector** (Figure 10). Each plan of the scenario is a button with a VALID or INVALID badge. Selecting a plan shows its totals (grid energy, total cost, peak grid import), the result of `sp_validate_plan`, a stacked chart of used solar, battery discharge and grid supply with the battery energy line, and the full 24-hour table from `v_plan_rows`. The button **+ Create baseline plan** calls `sp_create_baseline_plan`.

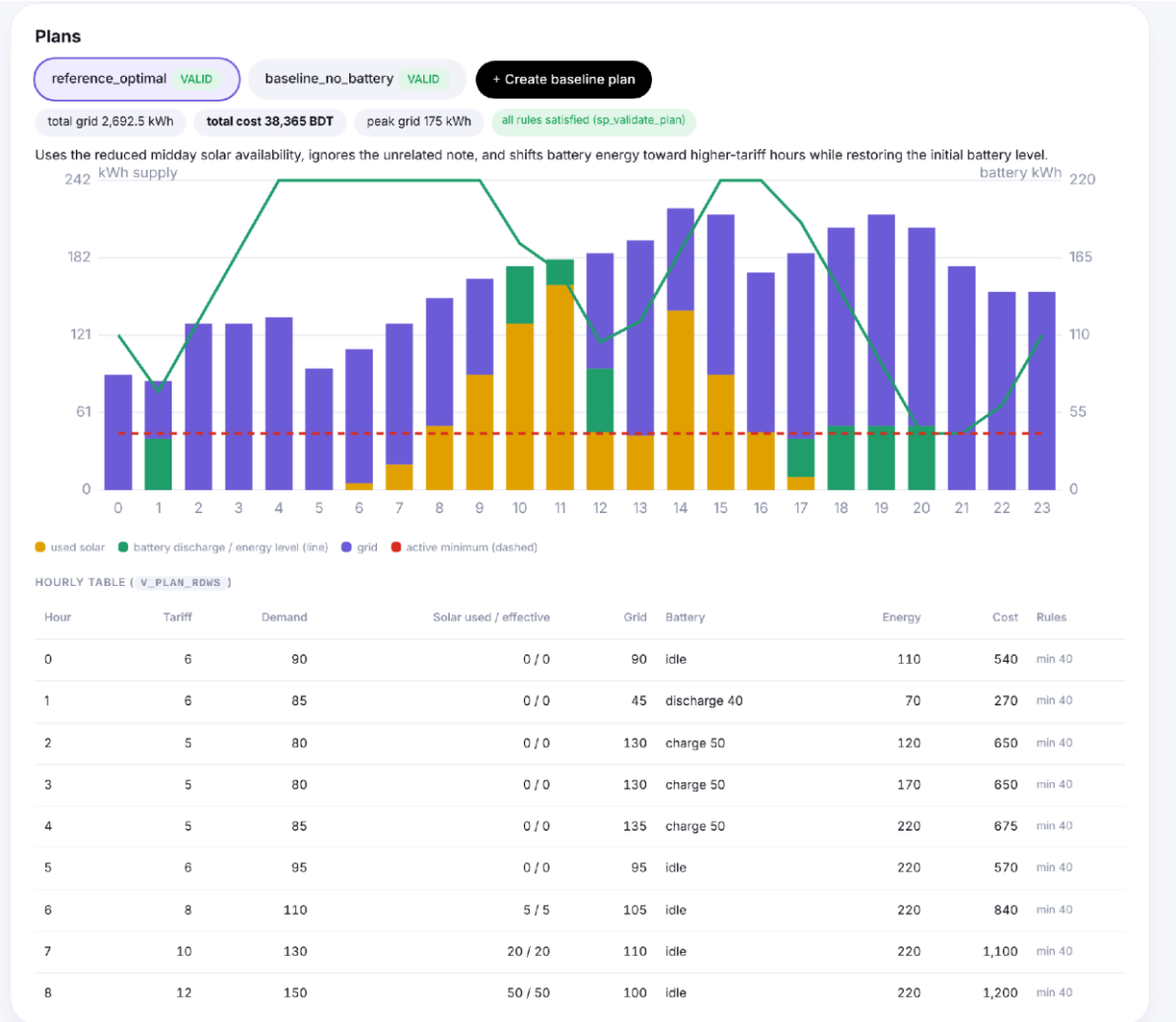


Figure 10: *Plan inspector with totals, hourly chart and rule check*

5.3.4 Import a Case (API Console)

The Import screen is an interactive console that posts a JSON case to the `import_case` endpoint. The three buttons are described in Table 21.

Table 21: *Buttons of the Import console*

Button	Action
Build a template	Fills the text box with a valid example case.
Template with a bad plan (trigger test)	Fills a case whose plan breaks a rule, to demonstrate the trigger.
Import	Sends the JSON to the API. A valid case is stored in one transaction.

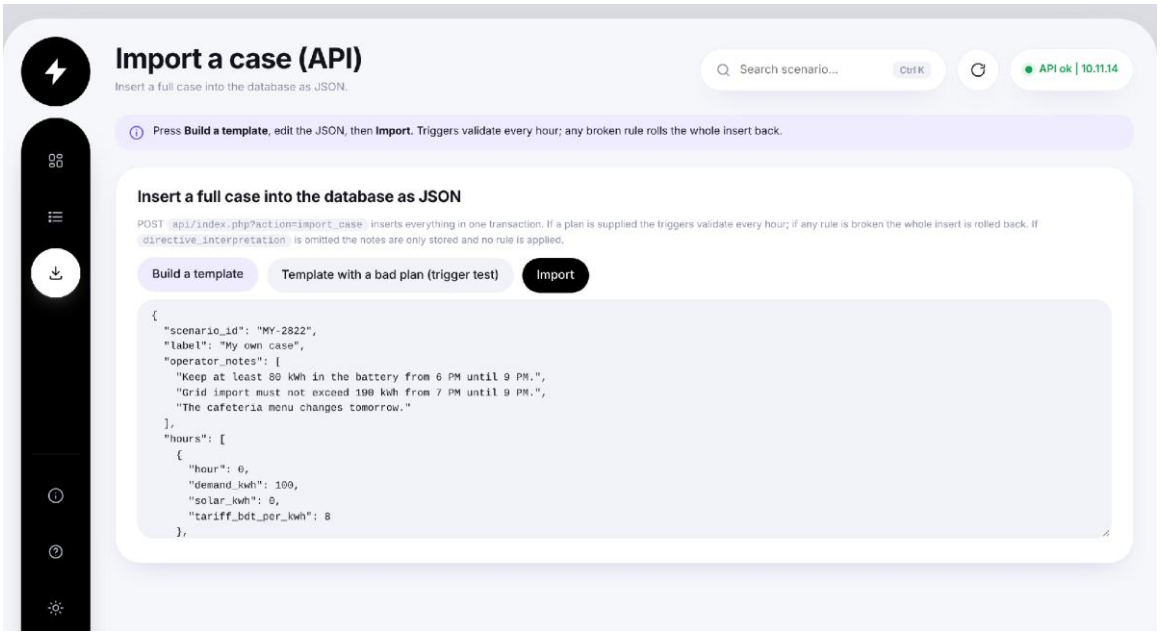


Figure 11: *Import console with a template case*

A valid case is stored and the console reports the new scenario (Figure 12). The stored case immediately appears in the Scenario list with source `user`.

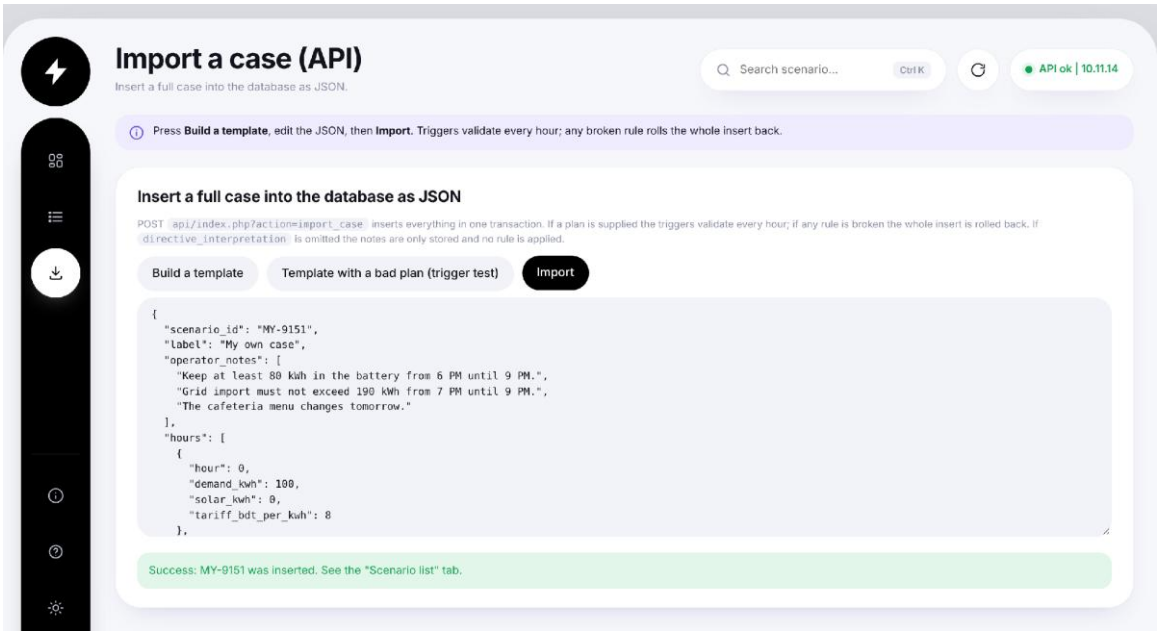


Figure 12: *Successful import of a valid case*

When the case contains a plan that breaks a rule, the BEFORE trigger raises SQLSTATE 45000. The API returns **HTTP 422** and the console shows the rule message in red; the whole transaction is rolled back and no table changes (Figure 13).

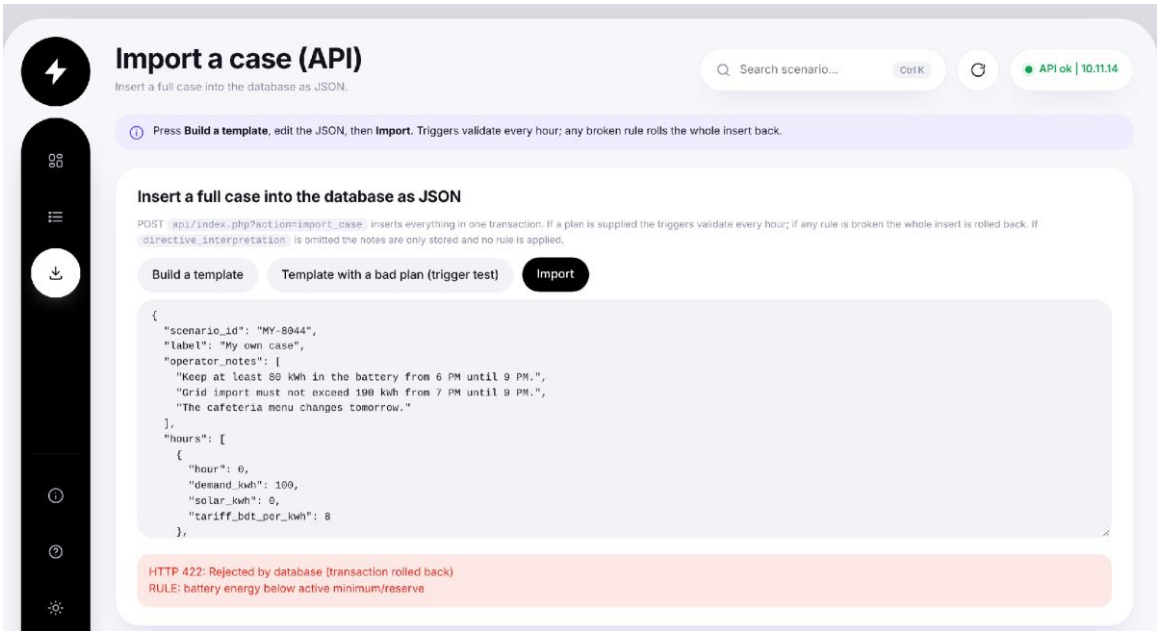


Figure 13: *Rejected case: trigger error and automatic rollback*

5.3.5 Help Panels and Themes

The sidebar button **How it works** opens a guide that explains every button of the interface (Figure 14). **About Project** shows the developer and system details. The **Light / Dark** button switches the theme and the choice is remembered by the browser (Figure 15). **Collapse menu** switches the sidebar between icons only and icons with labels.

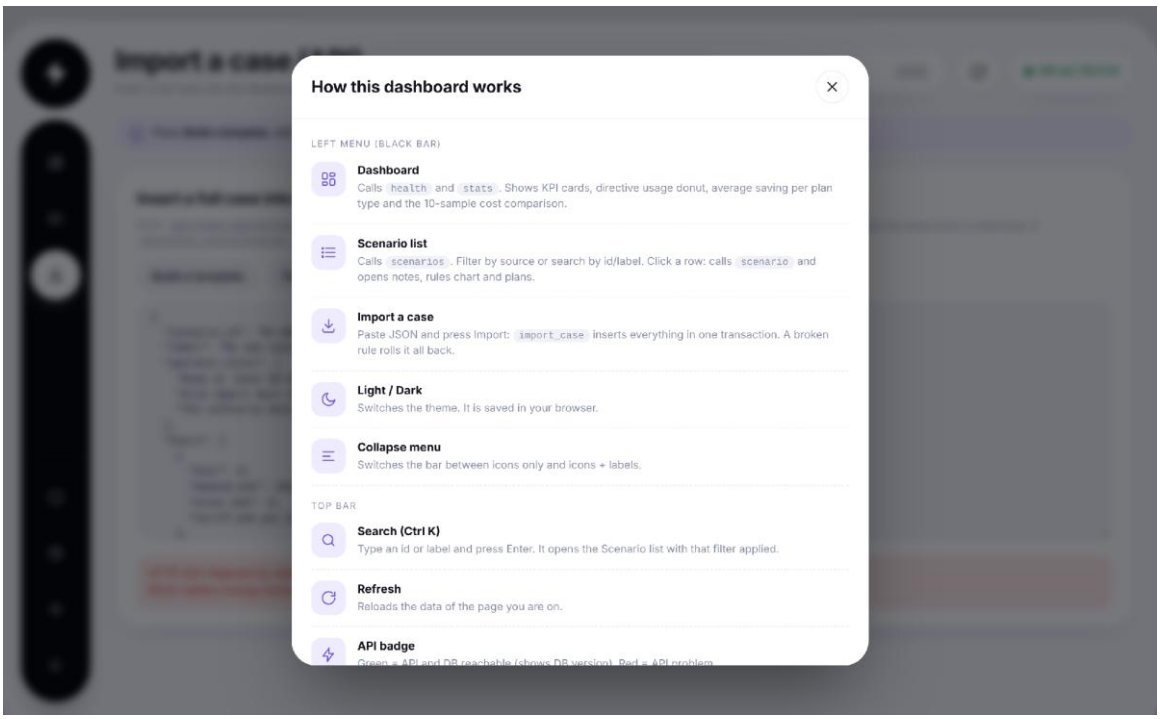


Figure 14: *The How it works guide panel*

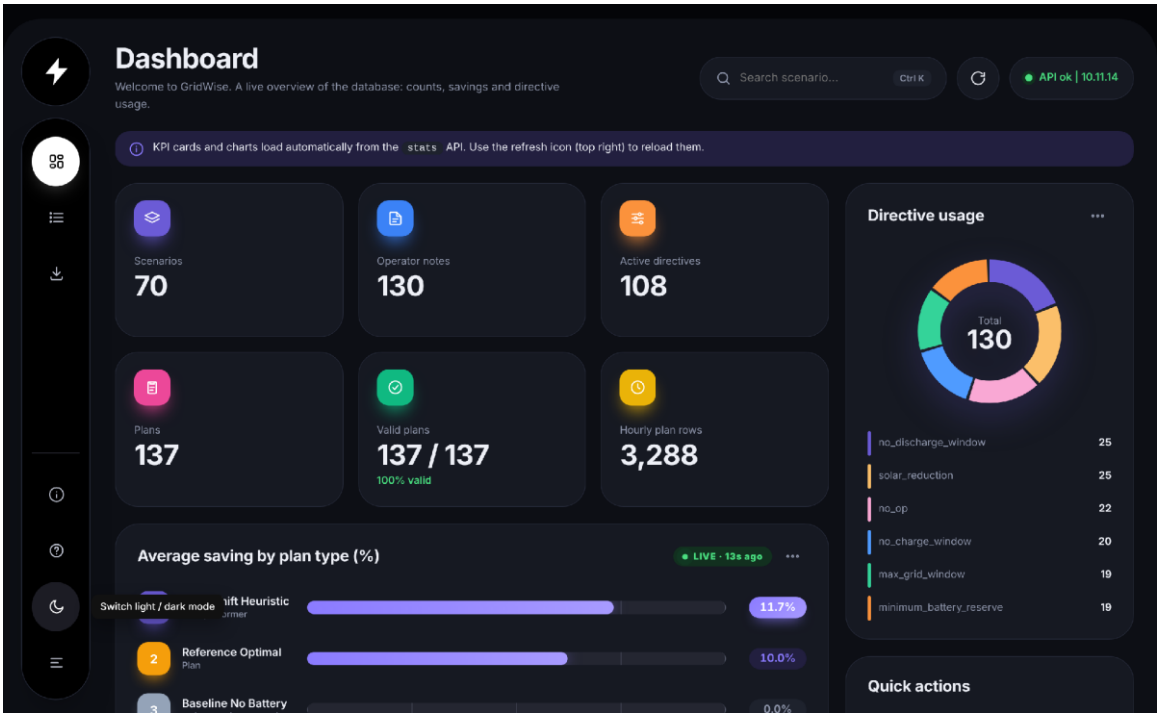


Figure 15: *Dashboard in dark theme*

5.3.6 Login Credentials

The dashboard and the API need no user login. The only credentials are those of the database connection configured in `api/db.php`: user `root` with an empty password (XAMPP default). If the local MySQL root account has a password, change it in `db.php`.

5.4 Temporary Live Preview for Remote Demonstration

For a remote viva the local dashboard can be opened temporarily on the internet. Keep Apache and MySQL running and start one of the following tunnels in a terminal.

Option 1 - Cloudflare Tunnel (cloudflared installed)

```
cloudflared tunnel --url http://localhost
```

Option 2 - install cloudflared first (Windows Package Manager)

```
winget install Cloudflare.cloudflared
```

Option 3 - localtunnel (Node.js)

```
npx localtunnel --port 80
```

The terminal prints a public HTTPS address. Append the dashboard path to it and share the result:

Address format

```
https://<generated-address>/gridwise/index.html
```

The terminal must stay open during the review; closing it (or pressing Ctrl+C) ends the public access. The tunnel exposes only the Apache web server; the MySQL port 3306 and the root credentials are not exposed.

5.5 Troubleshooting

Table 22: *Common problems and solutions*

Problem	Likely cause and solution
API badge is red	MySQL is not started or `api/db.php` has a wrong user or password. Start MySQL in XAMPP and check the connection settings.
Blank page or 404	Files are not in `C:\xampp\htdocs\gridwise\`. Copy the contents of `web/` again.
Import fails in phpMyAdmin	File larger than the upload limit. Import the five scripts 01 to 05 one by one, or use the command-line method.
`sp_selftest` does not exist	The installer runs only scripts 01 to 05. Import `07_selftest.sql` once.
Port 80 or 3306 already in use	Another program uses the port. Stop it or change the port in the XAMPP configuration.

CHAPTER 6
CONCLUSION

6.1 Conclusion

GridWise shows that the relational engine can be the active guardian of domain rules. The final system contains 10 BCNF tables, 11 views, 6 triggers, 7 stored procedures and 1 function, and stores 70 scenarios, 137 plans and 3,288 hourly records. All stored plans are checked by `v_plan_violations`, which returns an empty set, and an invalid row (for example, a solar reduction violation or a broken energy balance) is rejected at the database level regardless of whether it arrives from the web API, phpMyAdmin or a script. Table 23 compares the objectives of Section 1.2 with the measured results.

Table 23: *Objectives versus achieved results*

Objective	Result achieved
Normalized schema up to BCNF	10 tables in BCNF with declarative PK, FK, UNIQUE and CHECK constraints.
Rules enforced inside the DBMS	6 triggers and 7 procedures; rule-breaking rows rejected with SQLSTATE 45000.
Aggregates derived through views	11 views; no total is stored redundantly.
Automatic audit trail	AFTER UPDATE / DELETE triggers write to plan_audit.
Proof of correctness	18 of 18 self-tests passed; compliance audit returns 0 violations; rollback demonstration.
Advanced SQL demonstrated	26 analytical queries (joins, CTEs, window functions, subqueries, full-text) and an EXPLAIN index demo.
Web dashboard and REST API	PHP PDO API with 9 endpoints and a single-page dashboard with import console.

Main learning outcomes: designing and normalizing a real-world schema; expressing business rules as constraints, triggers and routines; deriving data through views instead of storing it; and combining a database with a lightweight PHP/JavaScript front end.

6.2 Limitations

- The system validates plans but does not compute optimal plans; plan generation is outside the scope.
- It models one battery per scenario and hourly time steps only.
- The web API has no authentication or role-based access and is intended for local or temporary demonstration use.
- Data are reference and synthetic cases; no live smart-meter or IoT data are connected.
- The target stack is XAMPP on a single machine; no replication, backup strategy or cloud deployment is included.

6.3 Future Works

- Integrate an optimization module (for example linear programming) that generates plans which the database then validates.
- Add user accounts, roles and permissions, and secure the REST API.
- Support multiple batteries, sub-hourly intervals and multi-site campuses.
- Ingest real meter and weather forecast data and add forecasting.
- Package the system with Docker and add a cloud database option and automated backups.
- Add more directive types and a natural-language parser for operator notes.

References

[1]GridWise Database System: source code and documentation. GitHub, https://github.com/SaKin32/Gridwise_db.

[2]Elmasri, R. and Navathe, S. B. *Fundamentals of Database Systems*. Pearson.

[3]Silberschatz, A., Korth, H. F. and Sudarshan, S. *Database System Concepts*. McGraw-Hill.

[4]Connolly, T. and Begg, C. *Database Systems: A Practical Approach to Design, Implementation, and Management*. Pearson.

[5]MySQL 8.0 Reference Manual: Triggers, Stored Routines, Window Functions and SIGNAL. https://dev.mysql.com/doc/refman/8.0/en/.

[6]MariaDB Knowledge Base. https://mariadb.com/kb/en/.

[7]PHP Manual: PHP Data Objects (PDO). https://www.php.net/manual/en/book.pdo.php.

[8]Apache Friends. XAMPP. https://www.apachefriends.org/.

[9]Codd, E. F. A Relational Model of Data for Large Shared Data Banks. *Communications of the ACM*, 13(6), 1970.

[10]Cloudflare. Quick Tunnels (cloudflared). https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/.

[11]Graphviz: Graph Visualization Software. https://graphviz.org/.

Appendix A: Project Files, Tests and Code

A.1 Project directory structure

```
Gridwise_db/
|-- .gitignore
|-- README.md                    technical manual
|-- docs/
|   |-- ER_and_normalization.md   design theory, 1NF-BCNF
|   |-- ER_diagram.mmd            Mermaid source
|   |-- ER_diagram.png            rendered ER diagram
|-- sql/
|   |-- 01_schema.sql             tables, constraints, indexes
|   |-- 02_views.sql              11 views
|   |-- 03_routines_triggers.sql  6 triggers, procedures, function
|   |-- 04_seed_samples.sql       10 public reference cases
|   |-- 05_seed_synthetic.sql     60 synthetic scenarios
|   |-- 06_queries.sql            Q01 - Q26
|   |-- 07_selftest.sql           sp_selftest (18 tests)
|   |-- 08_transaction_demo.sql   ACID / SAVEPOINT demo
|   |-- install_all.sql           installer (01 - 05)
|-- tools/
|   |-- build_seed.py             JSON samples to SQL
|   |-- gen_synthetic.py          synthetic scenario generator
|-- web/
    |-- index.html                dashboard
    |-- api/
        |-- db.php                PDO connection
        |-- index.php             REST API
```

A.2 Automated self-test cases (sp_selftest)

Run with the following command in phpMyAdmin or any MySQL client connected to `gridwise_db` (after importing `07_selftest.sql`). Table 24 lists the 18 tests exactly as implemented in the procedure.

SQL

```
CALL sp_selftest();
```

Table 24: *Self-test cases of sp_selftest*

No.	Test description	Expected outcome
1	valid row is accepted (SAMPLE-01, hour 0, idle)	no error (accepted)
2	continuity: rejected if it does not match the previous hour	error: transition wrong
3	solar_reduction: using more than the reduced solar is rejected	error: solar_used exceeds
4	rejected when energy balance is broken	error: energy balance
5	rejected when charge rate limit is exceeded	error: charge exceeds
6	rejected when discharge rate limit is exceeded	error: discharge exceeds
7	rejected when battery goes above capacity	error: above capacity
8	rejected when battery goes below the minimum	error: below active minimum
9	rejected when battery does not return to initial in the last hour	error: end-of-day
10	no_charge_window: charging in a forbidden hour is rejected	error: no_charge_window
11	no_discharge_window: discharging in a forbidden hour is rejected	error: no_discharge_window

No.	Test description	Expected outcome
12	max_grid_window: rejected when the grid cap is exceeded	error: max_grid_window
13	minimum_battery_reserve: rejected when going below the reserve	error: below active minimum
14	CHECK: battery_kwh > 0 while idle is rejected	error: ck_hplan_idle
15	guardrail: rejected when reserve exceeds capacity	error: reserve exceeds
16	CHECK: solar factor above 1 is rejected	error: ck_directive_factor
17	CHECK: applies=1 on no_op is rejected	error: ck_directive_applies
18	CHECK: giving a factor to no_charge_window is rejected	error: ck_directive_factor

A.3 Rule violation raised by the validator

Excerpt from the procedure `sp_check_plan_row` (file `sql/03_routines_triggers.sql`):

```
-- (2) energy balance: grid + solar + discharge = demand + charge
IF ABS((p_grid + p_solar + v_discharge) - (v_demand + v_charge)) > 0.01 THEN
  SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
    'RULE: energy balance broken (grid+solar+discharge <> demand+charge)';
END IF;
```

A.4 Selected analytical queries

```
-- Q16: cumulative cost per hour (window function)
SELECT hour_no, grid_kwh, tariff_bdt_per_kwh, hour_cost_bdt,
       SUM(hour_cost_bdt) OVER (ORDER BY hour_no) AS cumulative_cost_bdt
FROM v_plan_rows
WHERE scenario_id = 'SAMPLE-01' AND plan_name = 'reference_optimal'
ORDER BY hour_no;

-- Q17: reference plans costing more than the average (subquery)
SELECT scenario_id, total_cost_bdt
FROM v_plan_totals
WHERE plan_name = 'reference_optimal'
  AND total_cost_bdt > (SELECT AVG(total_cost_bdt) FROM v_plan_totals
                        WHERE plan_name = 'reference_optimal')
ORDER BY total_cost_bdt DESC;

-- Q22: universal compliance audit (expected: empty set)
SELECT * FROM v_plan_violations;
```

A.5 Transaction and rollback demonstration

Excerpt from `sql/08_transaction_demo.sql`. The valid edit is logged by the AFTER UPDATE trigger; ROLLBACK then restores the original data and the audit table is empty again.

```
START TRANSACTION;

-- valid edit (energy balance preserved): trigger accepts it
UPDATE hourly_plan hp JOIN plan p ON p.plan_id = hp.plan_id
   SET hp.solar_used_kwh = 80, hp.grid_kwh = 85
 WHERE p.scenario_id = 'SAMPLE-01' AND p.plan_name = 'baseline_no_battery'
   AND hp.hour_no = 9;

-- the AFTER UPDATE trigger logged the change
SELECT audit_id, plan_id, hour_no, action_type, old_grid_kwh, new_grid_kwh
FROM plan_audit;

SAVEPOINT before_bad_edit;
-- an invalid change (grid_kwh = 500) would raise:
--   RULE: energy balance broken ...
ROLLBACK TO SAVEPOINT before_bad_edit;

ROLLBACK;   -- cancel everything

-- verify: audit is empty, cost is as before
SELECT COUNT(*) AS audit_rows_after_rollback FROM plan_audit;
```

Appendix B: Data Dictionary

Column-level description of the 10 tables of `gridwise_db`, read directly from the installed database (`information_schema`).

Table 25: *Data dictionary of gridwise_db*

Column	Data type	Key	Null
scenario			
scenario_id	varchar(30)	PK	NOT NULL
label	varchar(120)		NOT NULL
source	enum('public_sample','synthetic','user')		NOT NULL
created_at	timestamp		NOT NULL
battery_config			
scenario_id	varchar(30)	PK	NOT NULL
capacity_kwh	decimal(10,2)		NOT NULL
initial_energy_kwh	decimal(10,2)		NOT NULL
minimum_energy_kwh	decimal(10,2)		NOT NULL
max_charge_kwh_per_hour	decimal(10,2)		NOT NULL
max_discharge_kwh_per_hour	decimal(10,2)		NOT NULL
hourly_input			
scenario_id	varchar(30)	PK	NOT NULL
hour_no	tinyint(3) unsigned	PK	NOT NULL
demand_kwh	decimal(10,2)		NOT NULL
solar_kwh	decimal(10,2)		NOT NULL
tariff_bdt_per_kwh	decimal(10,2)	Index	NOT NULL
operator_note			
note_id	int(10) unsigned	PK	NOT NULL
scenario_id	varchar(30)	FK / index	NOT NULL
note_index	tinyint(3) unsigned		NOT NULL
note_text	varchar(600)	FULLTEXT index	NOT NULL
directive_type			
type_code	varchar(30)	PK	NOT NULL
description	varchar(200)		NOT NULL
param_name	varchar(30)		NULL
directive			
directive_id	int(10) unsigned	PK	NOT NULL
note_id	int(10) unsigned	UNIQUE	NOT NULL
type_code	varchar(30)	FK / index	NOT NULL

Column	Data type	Key	Null
applies	tinyint(1)		NOT NULL
factor	decimal(5,4)		NULL
minimum_energy_kwh	decimal(10,2)		NULL
max_grid_kwh	decimal(10,2)		NULL
explanation	varchar(500)		NULL
directive_hour			
directive_id	int(10) unsigned	PK	NOT NULL
hour_no	tinyint(3) unsigned	PK	NOT NULL
plan			
plan_id	int(10) unsigned	PK	NOT NULL
scenario_id	varchar(30)	FK / index	NOT NULL
plan_name	varchar(60)		NOT NULL
plan_summary	varchar(700)		NULL
created_at	timestamp		NOT NULL
hourly_plan			
plan_id	int(10) unsigned	PK	NOT NULL
hour_no	tinyint(3) unsigned	PK	NOT NULL
grid_kwh	decimal(10,2)		NOT NULL
solar_used_kwh	decimal(10,2)		NOT NULL
battery_action	enum('charge','discharge','idle')	Index	NOT NULL
battery_kwh	decimal(10,2)		NOT NULL
energy_after_kwh	decimal(10,2)		NOT NULL
plan_audit			
audit_id	bigint(20) unsigned	PK	NOT NULL
plan_id	int(10) unsigned	FK / index	NOT NULL
hour_no	tinyint(3) unsigned		NOT NULL
action_type	enum('UPDATE','DELETE')		NOT NULL
old_grid_kwh	decimal(10,2)		NULL
new_grid_kwh	decimal(10,2)		NULL
old_action	varchar(10)		NULL
new_action	varchar(10)		NULL
changed_at	timestamp		NOT NULL